

LAPORAN PRAKTIKUM

Modul 4

Disusun Untuk Memenuhi Tugas Kuliah

Mata Kuliah: Teknologi IoT

Dosen Pembimbing:

Agus Ambarwari, S.Pd., M.Kom.

Dr. Ir. Septaliansyah Dwi Putra



Oleh:

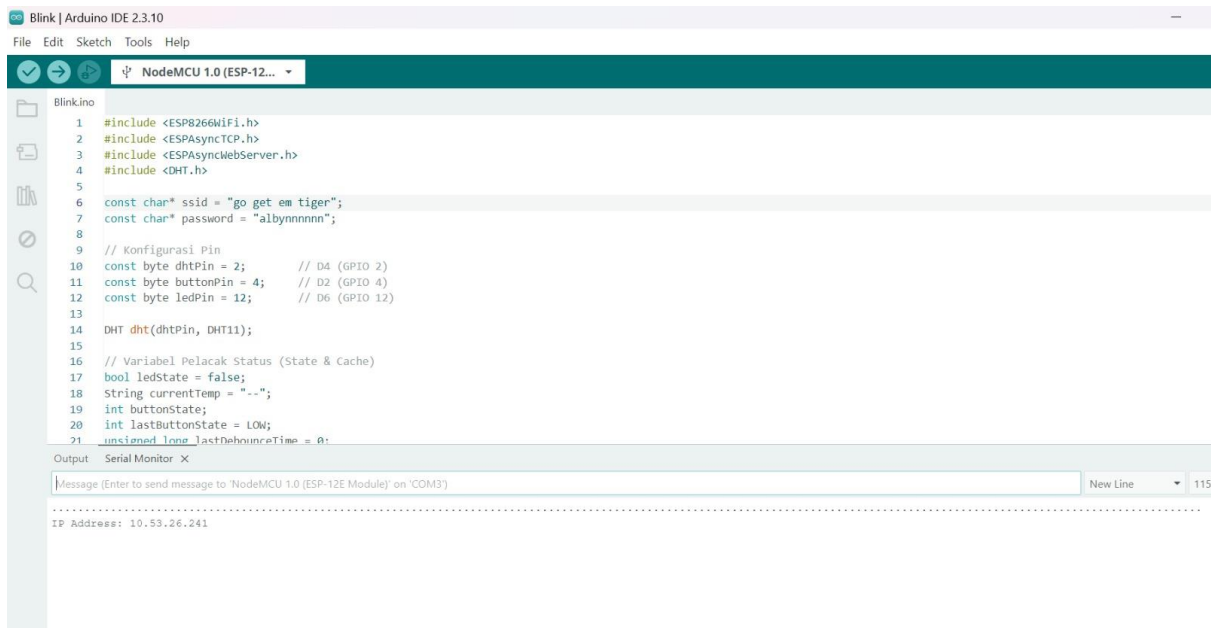
Alby Halim (25782031)

PROGRAM STUDI D4 TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGRILAMPUNG

4.5.1 Praktikum 1: Real-Time Dashboard & Sinkronisasi Tombol Fisik



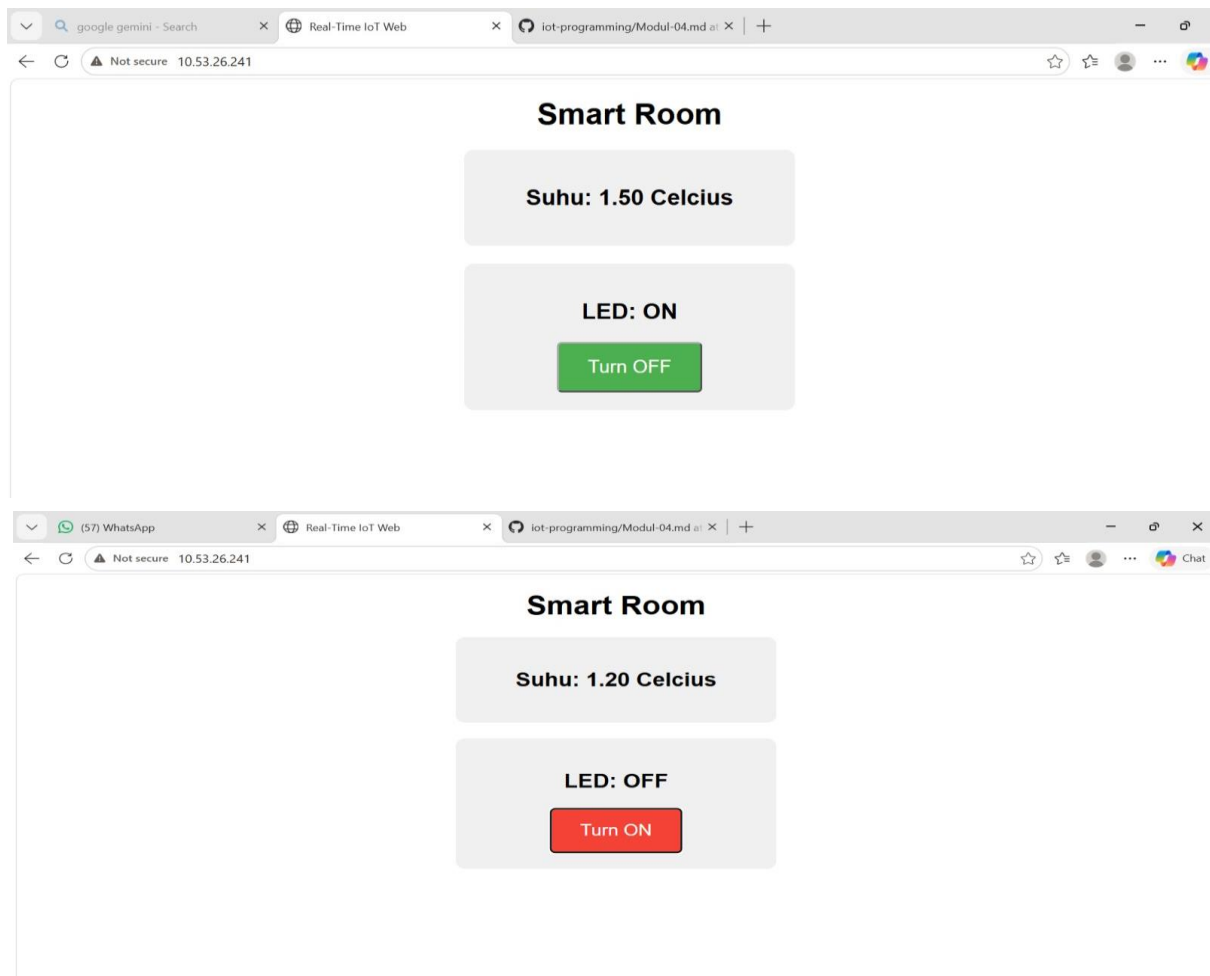
```
1 #include <ESP8266WiFi.h>
2 #include <ESPAsyncTCP.h>
3 #include <ESPAsyncWebServer.h>
4 #include <DHT.h>
5
6 const char* ssid = "go get em tiger";
7 const char* password = "albynntnnn";
8
9 // Konfigurasi Pin
10 const byte dhtPin = 2; // D4 (GPIO 2)
11 const byte buttonPin = 4; // D2 (GPIO 4)
12 const byte ledPin = 12; // D6 (GPIO 12)
13
14 DHT dht(dhtPin, DHT11);
15
16 // Variabel Pelacak Status (State & Cache)
17 bool ledState = false;
18 String currentTemp = "--";
19 int buttonState;
20 int lastButtonState = LOW;
21 unsigned long lastDebounceTime = 0;
```

Output Serial Monitor X

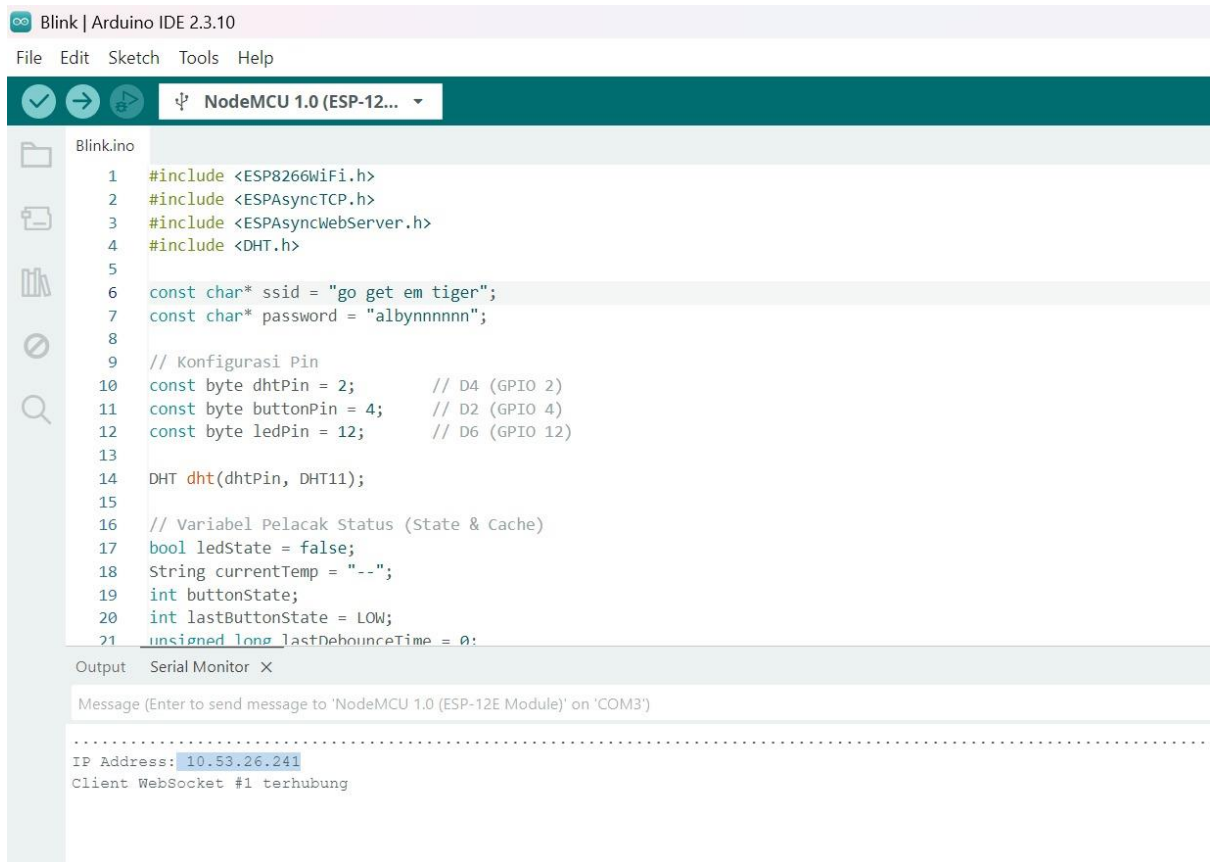
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

IP Address: 10.53.26.241

Disini terlihat Input utama untuk menjalankan kode utama pada modul 4 untuk bagian ssid dan password di sesuaikan dengan WiFi yang user gunakan, jika sudah connect akan muncul ip address dari koneksi WiFi lalu ip ini paste di browser



Jika sudah dijalankan di browser untuk ip tersebut akan muncul tampilan suhu dan user dapat memonitoring lampu LED dari jarak jauh



The screenshot shows the Arduino IDE interface. The top bar indicates 'Blink | Arduino IDE 2.3.10'. The menu bar includes 'File', 'Edit', 'Sketch', 'Tools', and 'Help'. The toolbar shows icons for saving, running, and uploading. The 'NodeMCU 1.0 (ESP-12...)' board is selected. The code editor displays the 'Blink.ino' file with the following code:

```
1 #include <ESP8266WiFi.h>
2 #include <ESPAsyncTCP.h>
3 #include <ESPAsyncWebServer.h>
4 #include <DHT.h>
5
6 const char* ssid = "go get em tiger";
7 const char* password = "albynnnnn";
8
9 // Konfigurasi Pin
10 const byte dhtPin = 2;      // D4 (GPIO 2)
11 const byte buttonPin = 4;   // D2 (GPIO 4)
12 const byte ledPin = 12;     // D6 (GPIO 12)
13
14 DHT dht(dhtPin, DHT11);
15
16 // Variabel Pelacak Status (State & Cache)
17 bool ledState = false;
18 String currentTemp = "--";
19 int buttonState;
20 int lastButtonState = LOW;
21 unsigned long lastDebounceTime = 0;
```

The Serial Monitor is open, showing the following output:

```
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')
.....
IP Address: 10.53.26.241
Client WebSocket #1 terhubung
```

Foto Rangkaian (LED ON)

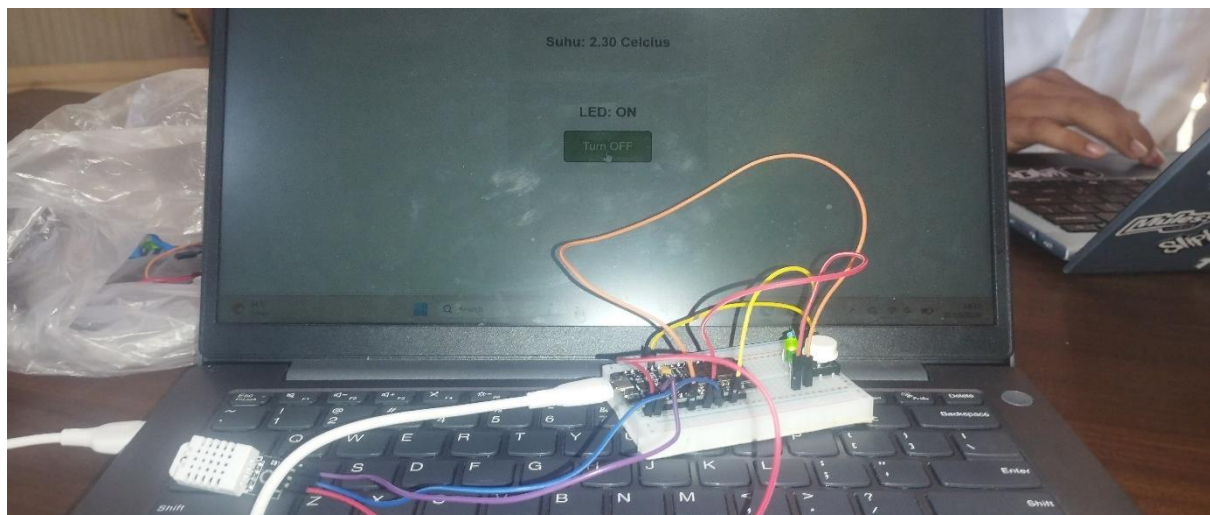
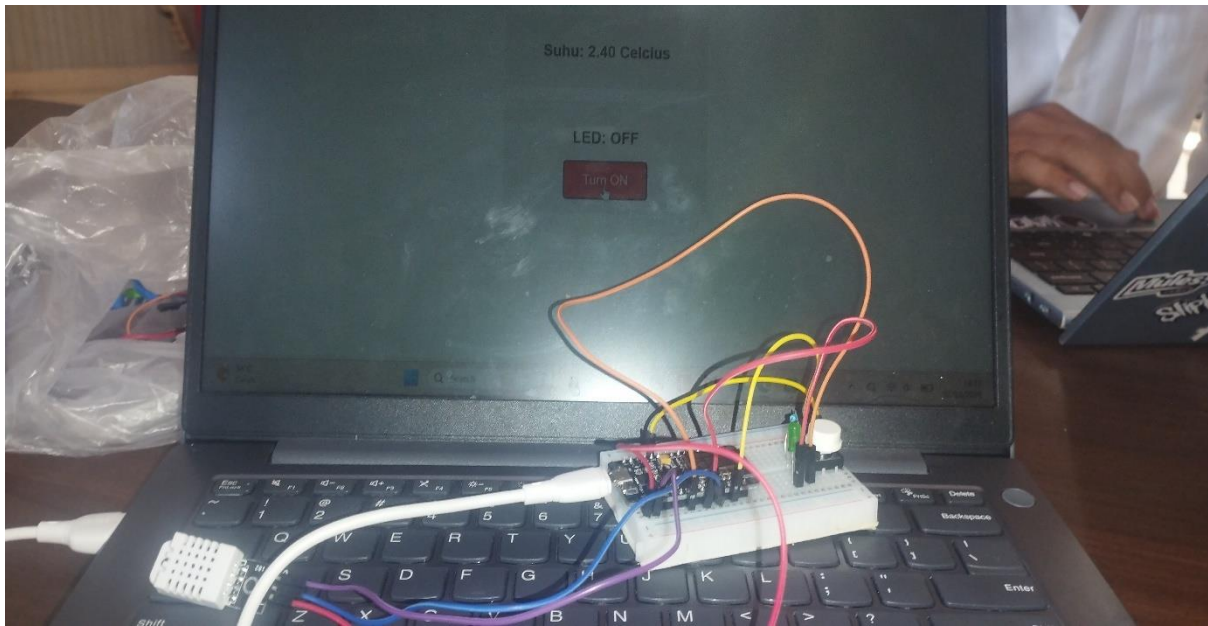


Foto Rangkaian (LED OFF)



Penjelasan Singkat Kode Khusus:

- `Ws.cleanupClients()`: (Berada di siklus `loop()`). Fungsi penting dari pustaka `Asynchronous` ini bertugas membersihkan dan menutup sesi (resource memori) dari klien/browser yang sudah terputus atau menutup tab layarnya. Tanpa baris ini, memori RAM ESP8266 akan penuh (memory leak) dan berujung pada Crash/Restart jika terlalu banyak perangkat keluar-masuk membuka halaman Dashboard.
- `pinMode(buttonPin, INPUT)` dan `if (buttonState == HIGH)`: Mempertahankan konfigurasi resistor Pull-Down eksternal 10k Ohm ke jalur GND (seperti pada Modul 1). Saat tombol dilepas, pin D2 membaca logika LOW (0V). Saat tombol ditekan, arus 3.3V mengalir sehingga terbaca logika HIGH yang memicu pembalikan status (toggle) LED secara instan.
- `If ((millis() - lastTime) > 3000)`: Pemrograman non-blocking. Alih-alih membuat CPU tidur selama 3 detik menggunakan `delay(3000)` (yang akan membekukan respons WebSocket), kita menggunakan selisih penghitung jam internal CPU (`millis()`). Dengan demikian, CPU dapat terus siaga mengawasi sentuhan tombol fisik dan interaksi jaringan internet dengan kecepatan tinggi.
- `Var dataObj = JSON.parse(event.data)`: Sintaks JavaScript yang membedah teks kaku kiriman C++ berformat `{"suhu": "30", "led": "1"}` menjadi struktur data objek dinamis (Dictionary) untuk memperbarui elemen desain HTML dengan instan (DOM Manipulation).
- `Ws.textAll()`: Perintah penyiaran (broadcasting) ampuh yang menembakkan/menyebarkan data secara seketika ke seluruh perangkat browser pengunjung (laptop, HP) secara bersamaan.

4.6 Latihan

Latihan 1 (Teori Jaringan): Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi WebSockets jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode Meta Refresh HTTP HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

Mengapa WebSockets Hemat Bandwidth:

Pada metode HTTP Meta Refresh, browser harus mengirimkan header request HTTP yang berukuran cukup besar (beberapa ratus byte hingga kilobyte) berulang kali hanya untuk meminta data kecil, lalu server merespons dengan mengirimkan seluruh dokumen HTML, CSS, dan JavaScript secara utuh. Sebaliknya, WebSockets menggunakan saluran komunikasi persistent/full-duplex (koneksi tetap terbuka). Setelah handshake awal selesai, data dikirim secara push oleh server hanya saat ada perubahan status/bacaan sensor dalam bentuk payload ringan (seperti string JSON berukuran belasan byte). Hal ini memangkas overhead header HTTP dan lalu lintas jaringan secara signifikan.

Mengapa Tidak Berkedip (No Flickering):

HTTP Meta Refresh memaksa browser untuk menghancurkan (unload) halaman lama dan memuat ulang (reload) seluruh struktur DOM halaman web dari awal. Sedangkan WebSockets beroperasi secara asinkron di belakang layar. Saat data diterima oleh JavaScript, browser hanya memperbarui teks elemen HTML tertentu (melalui DOM Manipulation) tanpa memuat ulang kerangka halaman web.

Latihan 2 (Analisis Waktu/Timer): Apa bahaya sistematis yang akan terjadi pada komunikasi WebSockets (reaksi antara browser dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks `delay(3000)` di dalam siklus fungsi `void loop()` pada draft arsitektur kompleks ini?.

Penggunaan `delay(3000)` di dalam `loop()` akan menghentikan (blocking) seluruh eksekusi CPU NodeMCU selama 3 detik penuh. Bahaya sistematis yang terjadi meliputi: Kehilangan Respons Tombol Fisik (Lag/Unresponsive): Pembacaan push button dan algoritma debounce tidak akan dieksekusi selama interval 3 detik tersebut. Jika tombol ditekan saat CPU sedang mengalami `delay()`, tekanannya tidak akan terdeteksi.

Koneksi WebSocket Terputus (Timeout/Disconnect): Pustaka `ESPAsyncWebServer` membutuhkan rutinitas berkala untuk mengelola ping/pong dan cleanup klien. Memblokir CPU akan menyebabkan terputusnya koneksi WebSocket karena server dianggap tidak merespons oleh browser (terjadi lag atau reconnect berulang).

Penyebab Latensi Tinggi: Perintah toggle dari web browser yang dikirimkan ke NodeMCU akan tertunda hingga `delay(3000)` selesai dijalankan.

Latihan 3 (Modifikasi Front-End): Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML () apa saja yang menjadi target injeksi/pengubahan teks oleh script yang ditulis oleh Anda?

Berdasarkan analisis fungsi JavaScript pada blok pembacaan paket JSON: tempValue (): Menjadi target injeksi nilai angka suhu hasil pembacaan sensor DHT11.

ledStatus (): Menjadi target pengubahan teks indikator status LED (ON atau OFF).

toggleBtn (<button id=”toggleBtn”>): Elemen tombol HTML yang diubah teks labelnya (Turn ON / Turn OFF) serta nama class CSS-nya (btn-on / btn-off).

Latihan 4 (Pengembangan Telemetry Sensor – Kelembapan): Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (temperature). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (humidity) secara real-time via WebSockets:

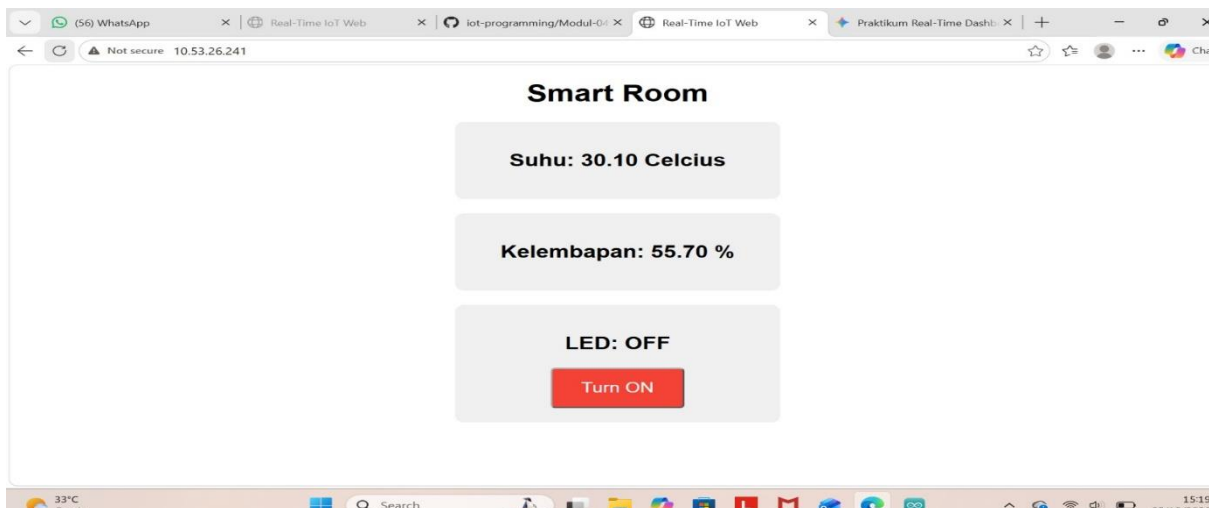
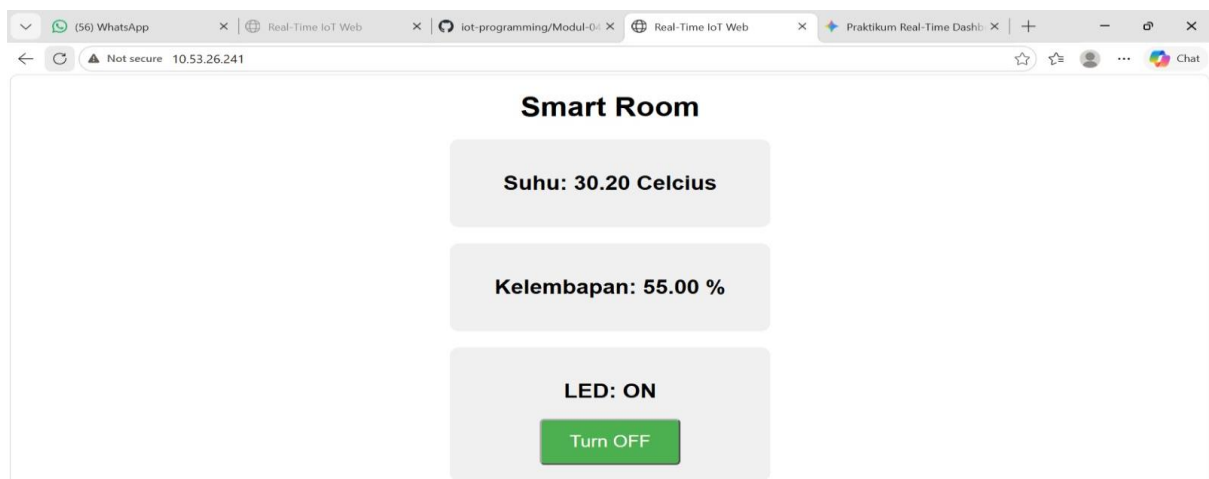
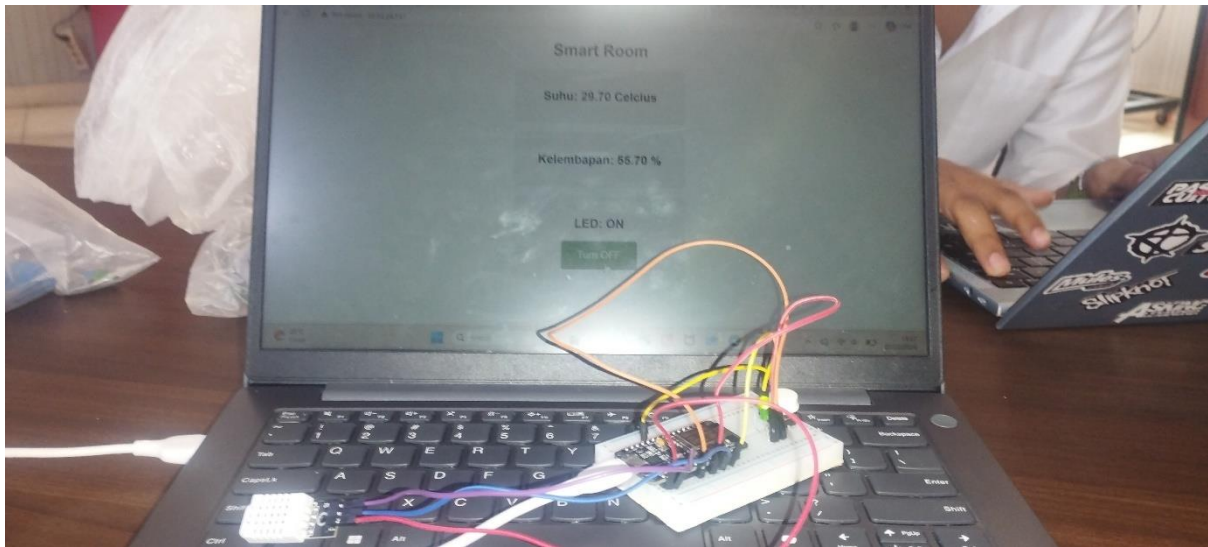


Foto Rangkaian



Latihan 4 pada Praktikum 4.5.1 bertujuan untuk menambahkan fitur pemantauan kelembapan udara secara real-time pada dashboard web dengan memanfaatkan protokol WebSockets. Pada sisi backend (C++), NodeMCU membaca data suhu dan kelembapan dari sensor DHT11 setiap 3 detik menggunakan timer non-blocking `millis()`, lalu mengemasnya ke dalam format JSON `{“led”:”...”, “suhu”:”...”, “hum”:”...”}` untuk dikirimkan secara otomatis ke seluruh browser. Sementara itu, pada sisi frontend (HTML & JavaScript), ditambahkan kartu baru `<span id=”humValue”>` yang nilainya diperbarui secara dinamis melalui teknik DOM manipulation tanpa perlu memuat ulang (refresh) halaman web.